

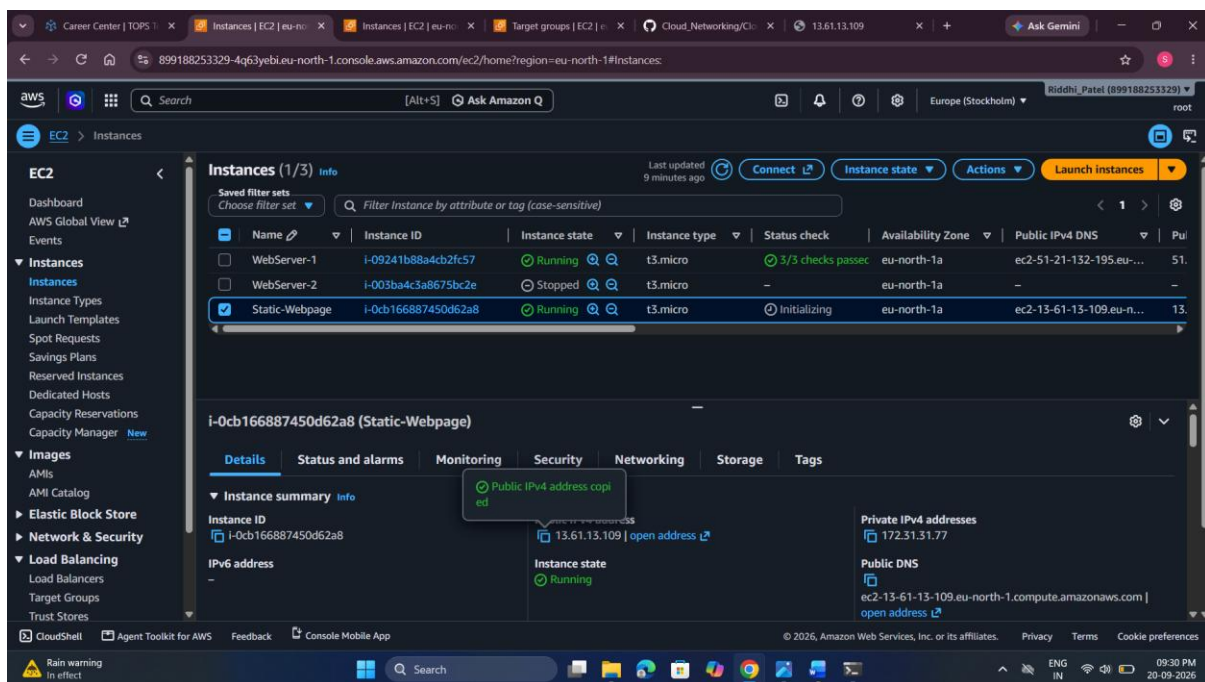
Cloud Networking – Assignment 3

1. Set up a free-tier AWS account (or use an AWS Cloud sandbox) and launch a basic EC2 instance that serves a static HTML page. Document the steps you took and the public URL of your instance.

Ans:

Step 1 : Launch an EC2 Instance The Amazon EC2 service was opened from the AWS Management Console. A new EC2 instance named Static-Web-Server was launched using an Amazon Linux AMI and a free-tier eligible instance type.

Screenshot 1: EC2 Instance Static-Webpage is Running



Step 2 : Configure Security Group

The Security Group was configured to allow HTTP traffic on port 80 from the internet. This allows users to access the web server through a browser.

Step 3: Install and Start Apache Web Server

The EC2 instance was connected using SSH through the Windows CMD.

Run : `ssh -i "My-web-server-key.pem" ec2-user@13.61.13.109`

The Apache web server was installed and started using:

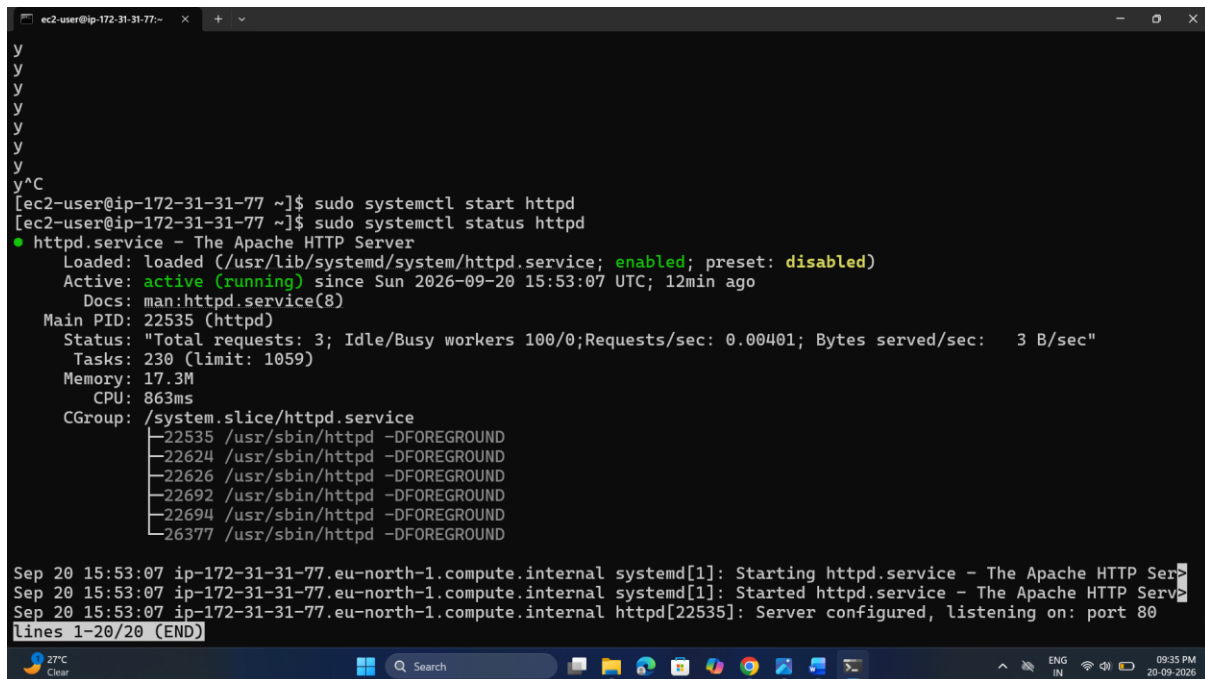
`sudo yum install -y httpd`

`sudo systemctl start httpd`

`sudo systemctl enable httpd`

`sudo systemctl status httpd`

Screenshot 2: Apache web server running



```
ec2-user@ip-172-31-31-77:~$ sudo systemctl start httpd
[ec2-user@ip-172-31-31-77 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-09-20 15:53:07 UTC; 12min ago
     Docs: man:httpd.service(8)
  Main PID: 22535 (httpd)
    Status: "Total requests: 3; Idle/Busy workers 100/0;Requests/sec: 0.00401; Bytes served/sec: 3 B/sec"
    Tasks: 230 (Limit: 1059)
   Memory: 17.3M
      CPU: 863ms
   CGroup: /system.slice/httpd.service
           └─22535 /usr/sbin/httpd -DFOREGROUND
             └─22624 /usr/sbin/httpd -DFOREGROUND
               └─22626 /usr/sbin/httpd -DFOREGROUND
                 └─22692 /usr/sbin/httpd -DFOREGROUND
                   └─22694 /usr/sbin/httpd -DFOREGROUND
                     └─26377 /usr/sbin/httpd -DFOREGROUND

Sep 20 15:53:07 ip-172-31-31-77.eu-north-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Ser
Sep 20 15:53:07 ip-172-31-31-77.eu-north-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Ser
Sep 20 15:53:07 ip-172-31-31-77.eu-north-1.compute.internal httpd[22535]: Server configured, listening on: port 80
lines 1-20/20 (END)
```

Step 4: Create the Static HTML Page

Run : `echo "<html><body><h1>Welcome to My static Webpage</h1></body></html>" | sudo tee /var/www/html/index.html`

This creates the HTML page that Apache will serve.

Step 5 : Open the Public URL

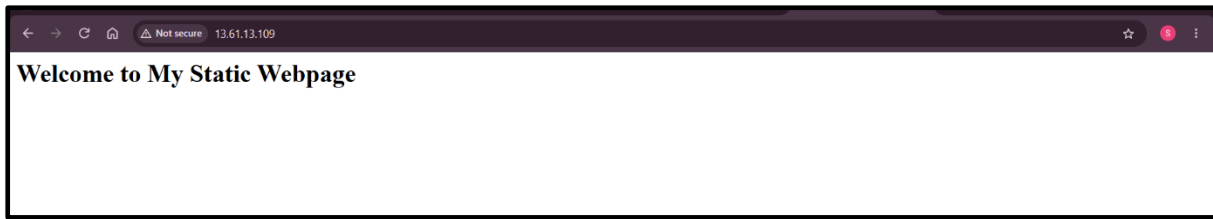
The Public IPv4 address of the EC2 instance was used in a web browser with HTTP:

<http://13.61.13.109/>

The static HTML page was successfully displayed in the browser

Public URL: <http://13.61.13.109/>

Screenshot 3: Static HTML page opened using the EC2 public IP



2. Create an Auto Scaling Group in AWS that maintains a minimum of 1 and a maximum of 3 EC2 instances for your web server. Set the scaling policy to add a new instance if CPU utilization goes above 60% for 5 minutes. Hint: Use the AWS Management Console's Auto Scaling Groups section and attach your EC2 launch template or configuration.

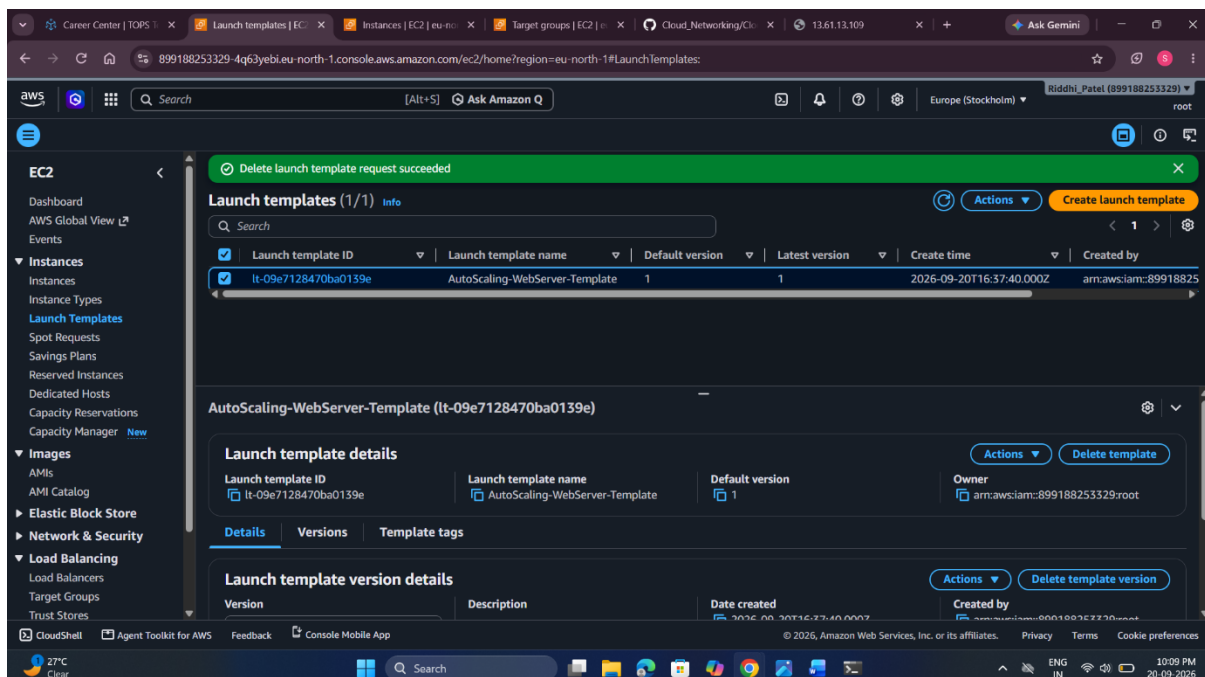
Ans : I created an Auto Scaling Group (ASG) for the web server using an EC2 Launch Template. The Auto Scaling Group was configured to maintain a minimum of 1 instance, a desired capacity of 1 instance, and a maximum of 3 instances.

A CloudWatch alarm was configured to monitor the average CPU utilization. If the CPU utilization goes above 60% for 5 minutes, the alarm triggers the scaling policy. The scaling policy then adds 1 EC2 instance to the Auto Scaling Group.

Step 1: Create a Launch Template

I created a launch template named **AutoScaling-WebServer-Template** for the web server. The template contains the AMI, instance type, and security group settings required to launch EC2 instances.

Screenshot 1: Launch Template – AutoScaling-WebServer-Template



Step 2: Configure Auto Scaling Group

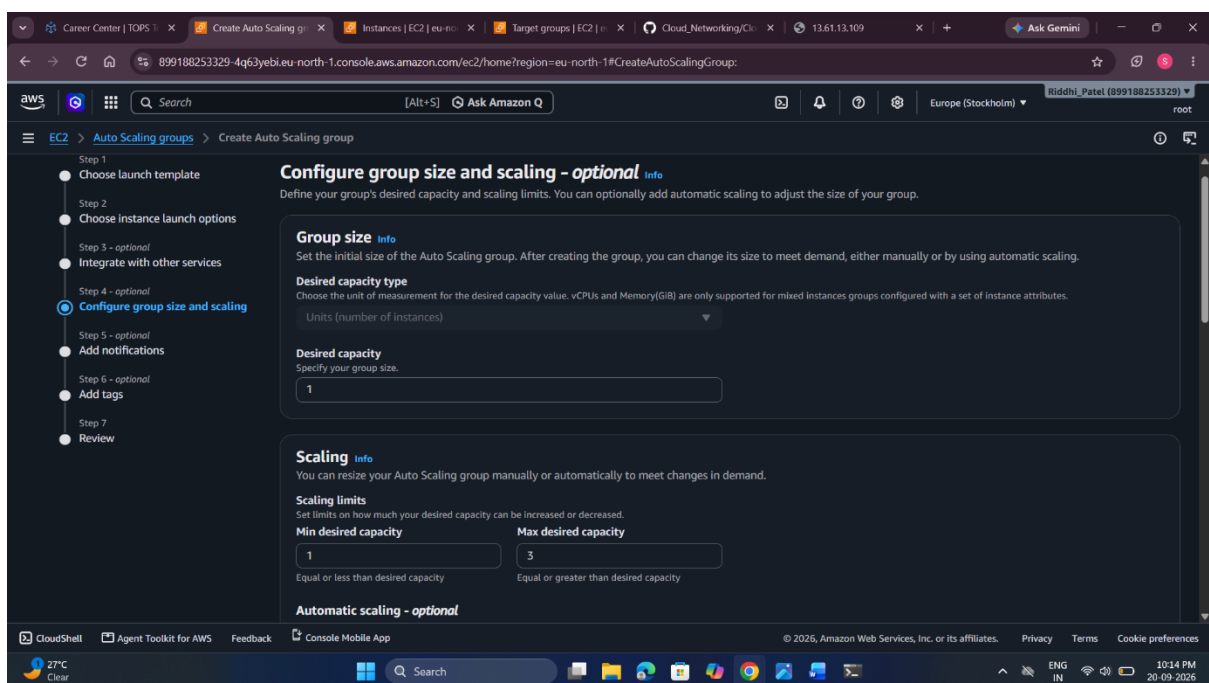
I created an Auto Scaling Group named **AutoScaling-WebServer-ASG** and selected the launch template.

The group size was configured as:

- **Minimum capacity: 1**
- **Desired capacity: 1**
- **Maximum capacity: 3**

The Auto Scaling Group can therefore maintain at least one instance and can increase the number of instances up to three when required.

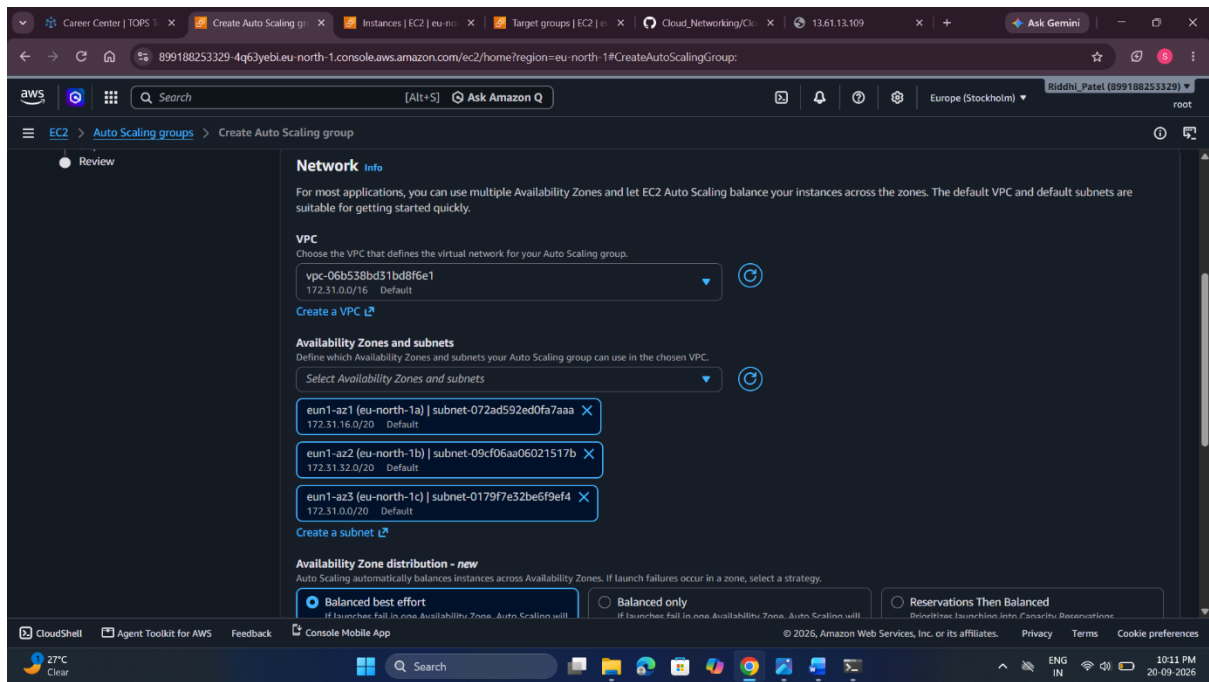
Screenshot 2: Auto Scaling Group Size Configuration



Step 3: Select Network

I selected the default VPC and the available subnets/Availability Zones for the Auto Scaling Group.

Screenshot 3: VPC and Availability Zones Configuration



Step 4: Configure CPU Monitoring

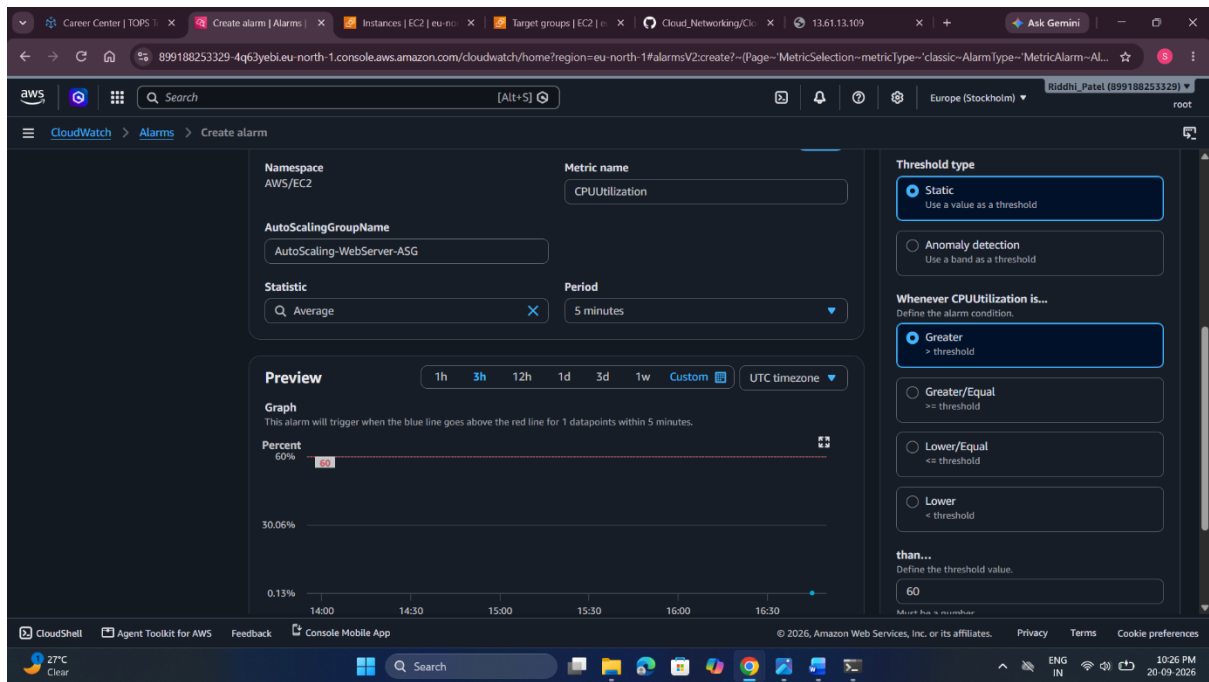
I created a CloudWatch alarm using the **AWS/EC2 CPUUtilization** metric for the Auto Scaling Group.

The alarm was configured as:

- **Metric:** CPUUtilization
- **Statistic:** Average
- **Period:** 5 minutes
- **Condition:** Greater than 60%
- **Evaluation:** 1 out of 1 period

Therefore, the alarm is triggered when the average CPU utilization goes above **60% for a 5-minute period**.

Screenshot 4: CloudWatch CPU Utilization Alarm

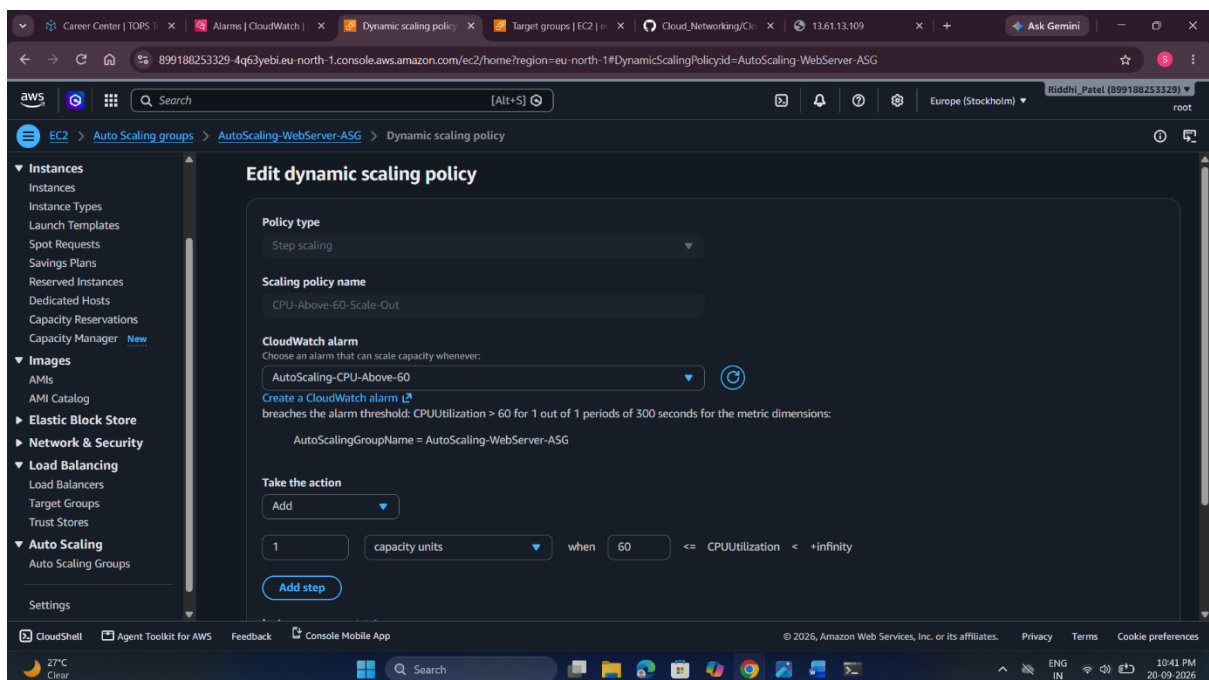


Step 5: Configure Scaling Policy

I created a **Step Scaling** policy named **CPU-Above-60-Scale-Out**.

The policy is connected to the CloudWatch alarm **AutoScaling-CPU-Above-60**. When the alarm is triggered, the policy adds **1 capacity unit**, which means one additional EC2 instance.

Screenshot 5: Step Scaling Policy Configuration

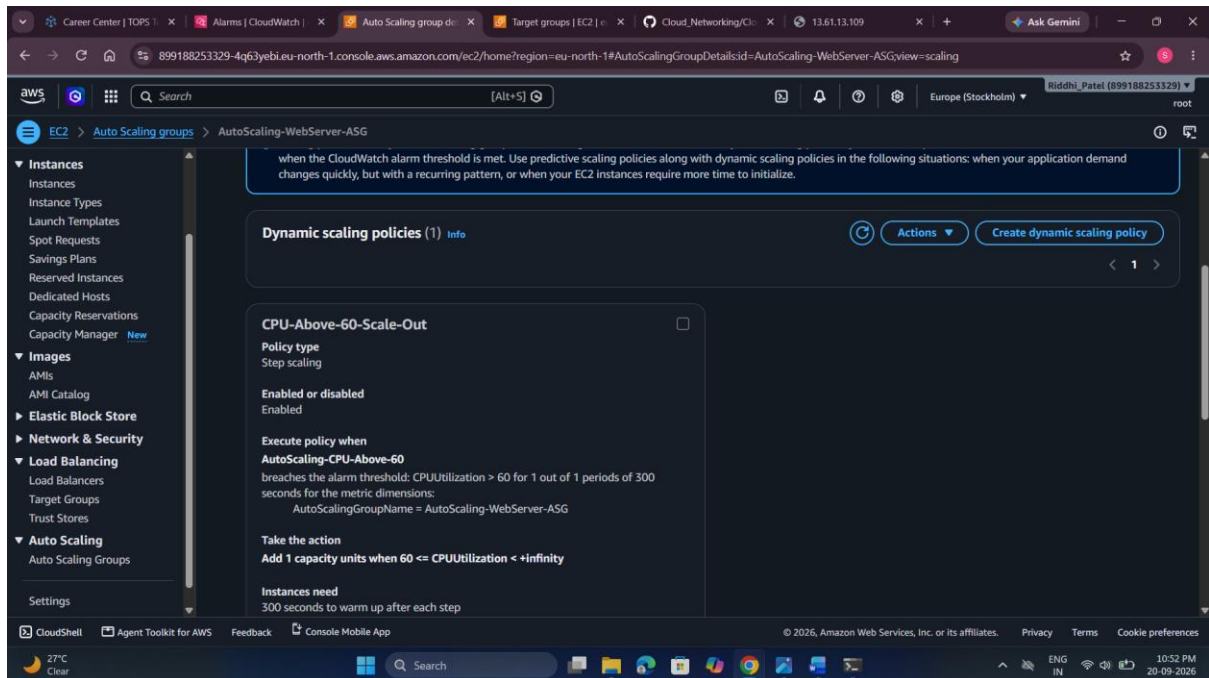


Step 6: Verify Scaling Policy

The Auto Scaling Group shows the dynamic scaling policy **CPU-Above-60-Scale-Out** as enabled.

Screenshot 6: Auto Scaling Group – Dynamic Scaling Policy

It shows the Auto Scaling Group and the successfully created dynamic scaling policy.

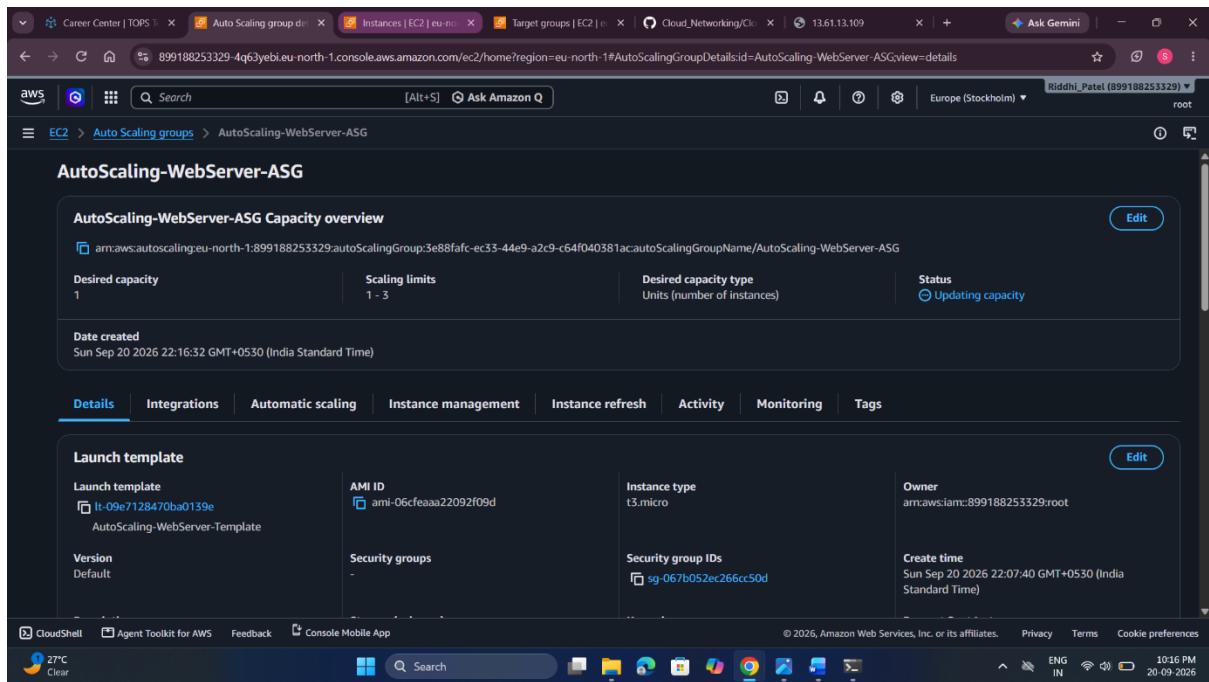


Step 7: Verify Auto Scaling Group

Finally, I verified the Auto Scaling Group configuration. The group has:

- Desired capacity = **1**
- Minimum = **1**
- Maximum = **3**
- Current instance = **1**
- Instance health = **1/1 Healthy**

Screenshot 7: Auto Scaling Group Capacity Overview



Result

The Auto Scaling Group was successfully configured with a **minimum of 1 and maximum of 3 EC2 instances**. A CloudWatch alarm monitors CPU utilization and triggers when CPU usage goes above **60% for 5 minutes**. The scaling policy then **adds 1 EC2 instance** to handle the increased workload.

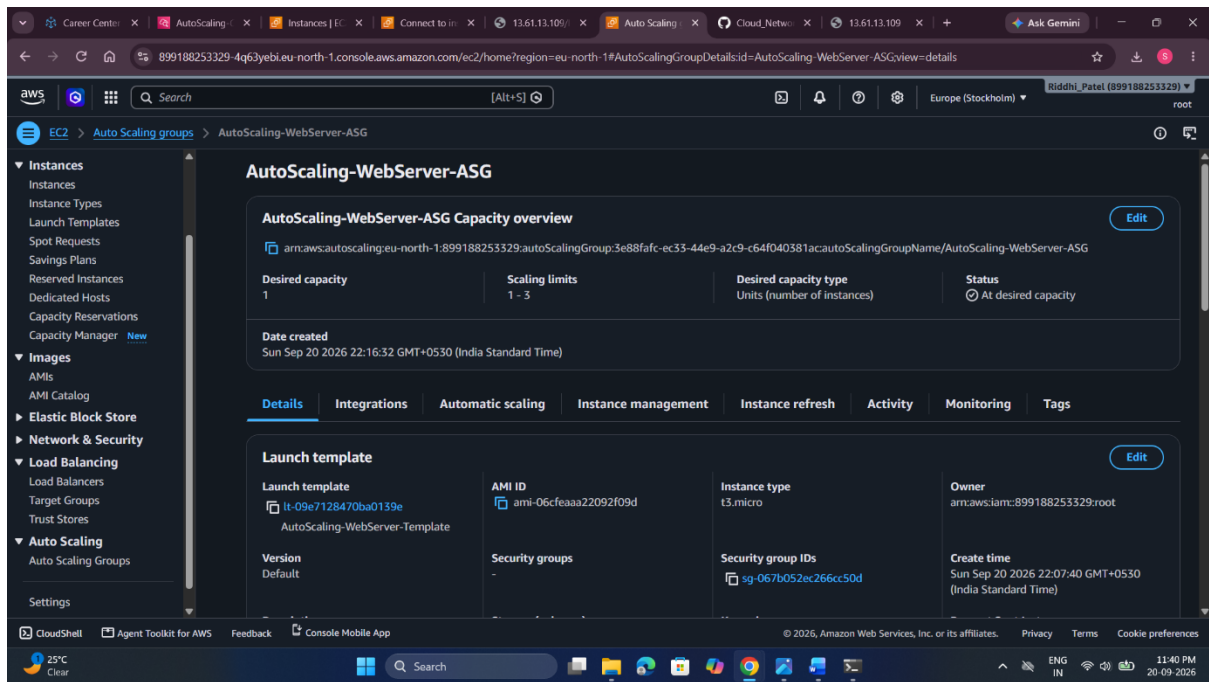
3. Simulate increased traffic to your web server by using an online load testing tool (such as Loader.io or k6) and observe if the Auto Scaling Group launches additional instances when CPU usage spikes.

Ans:

Step 1 :

The **Auto Scaling Group AutoScaling-WebServer-ASG** was checked before starting the test. It had **1 desired instance**, with a **minimum of 1 and maximum of 3 instances**.

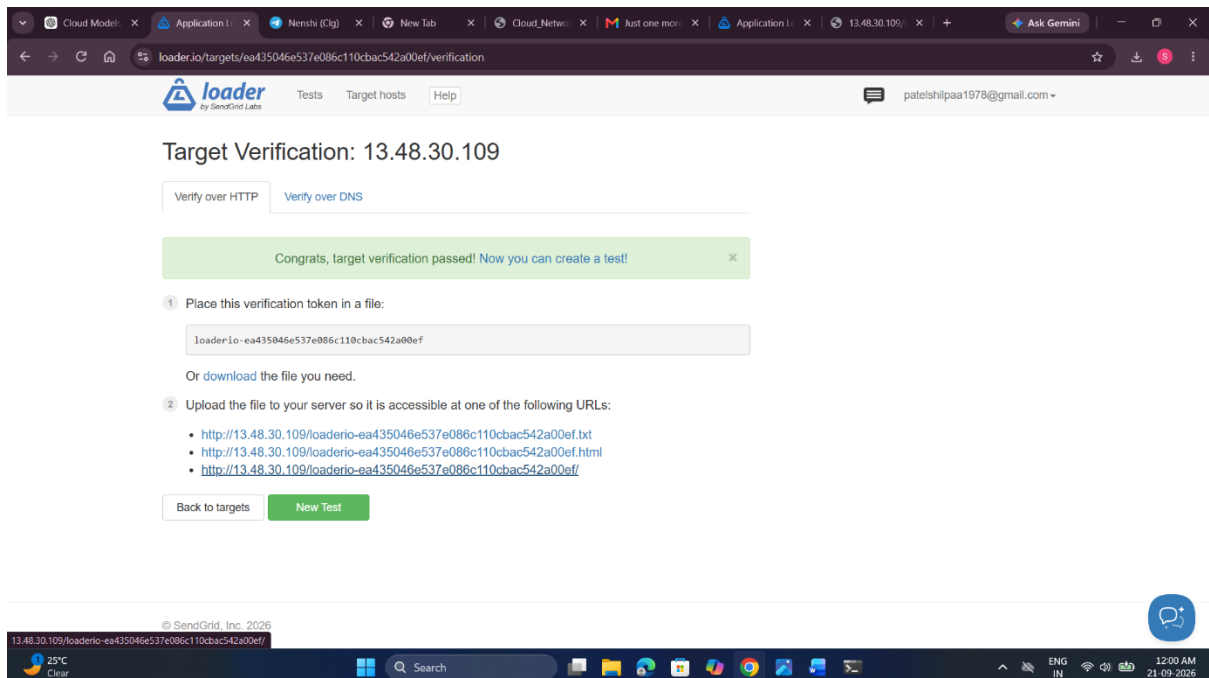
Screenshot 1: Auto Scaling Group Before Load Test



Step 2:

The ASG instance with public IP **13.48.30.109** was added as a target host in **Loader.io** and the target verification was completed successfully.

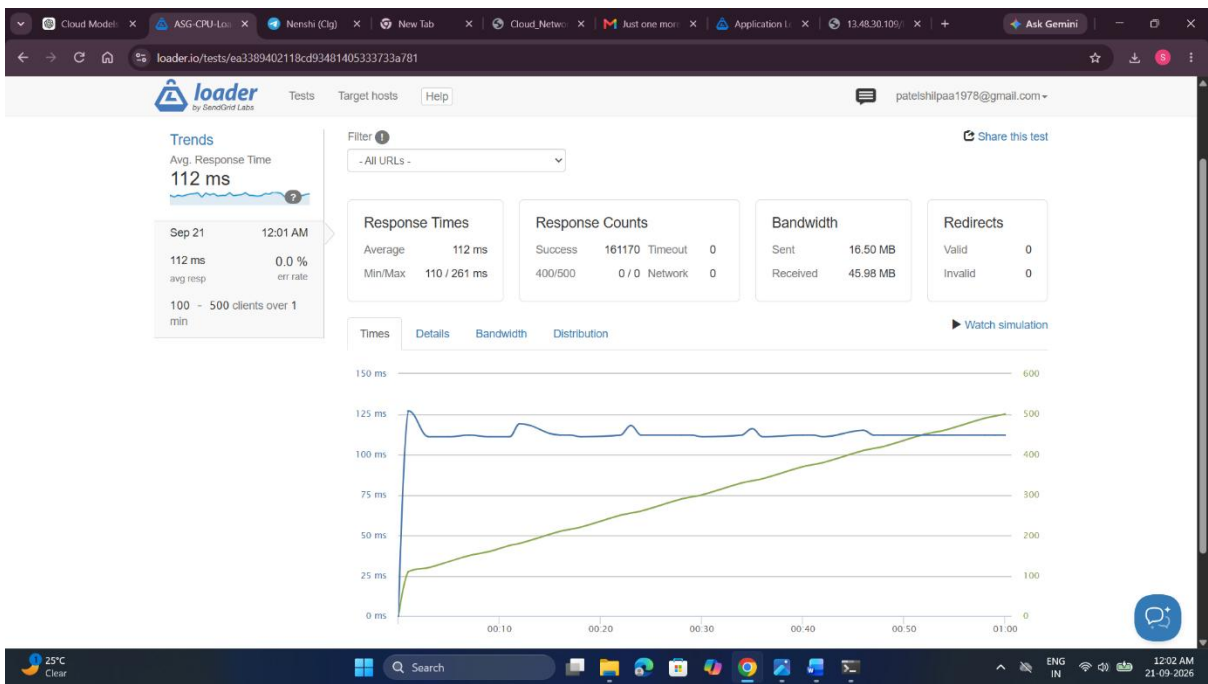
Screenshot 2: Loader.io Target Verification



Step 3:

A Loader.io load test was created with **100 to 500 clients over 1 minute** to generate increased traffic.

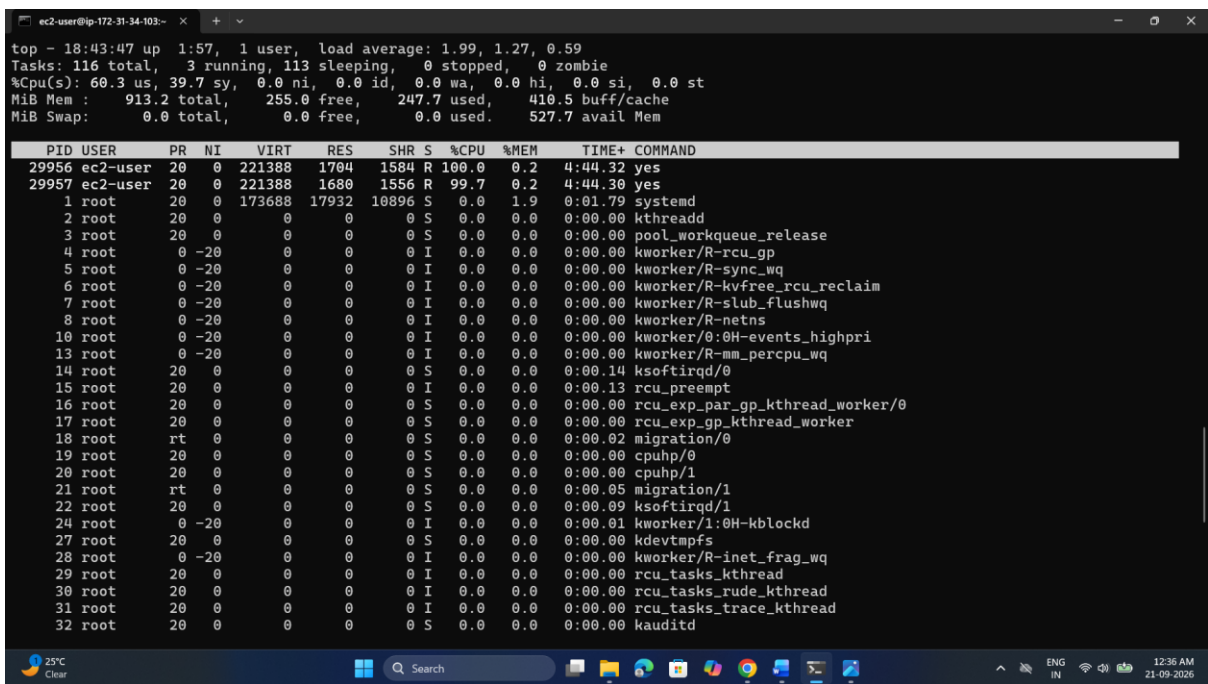
Screenshot 3: Loader.io Load Test Results



Step 4:

The CPU load on the ASG instance was increased to simulate a high workload. The CPU utilization reached approximately **99.99%**, crossing the configured **60% threshold**.

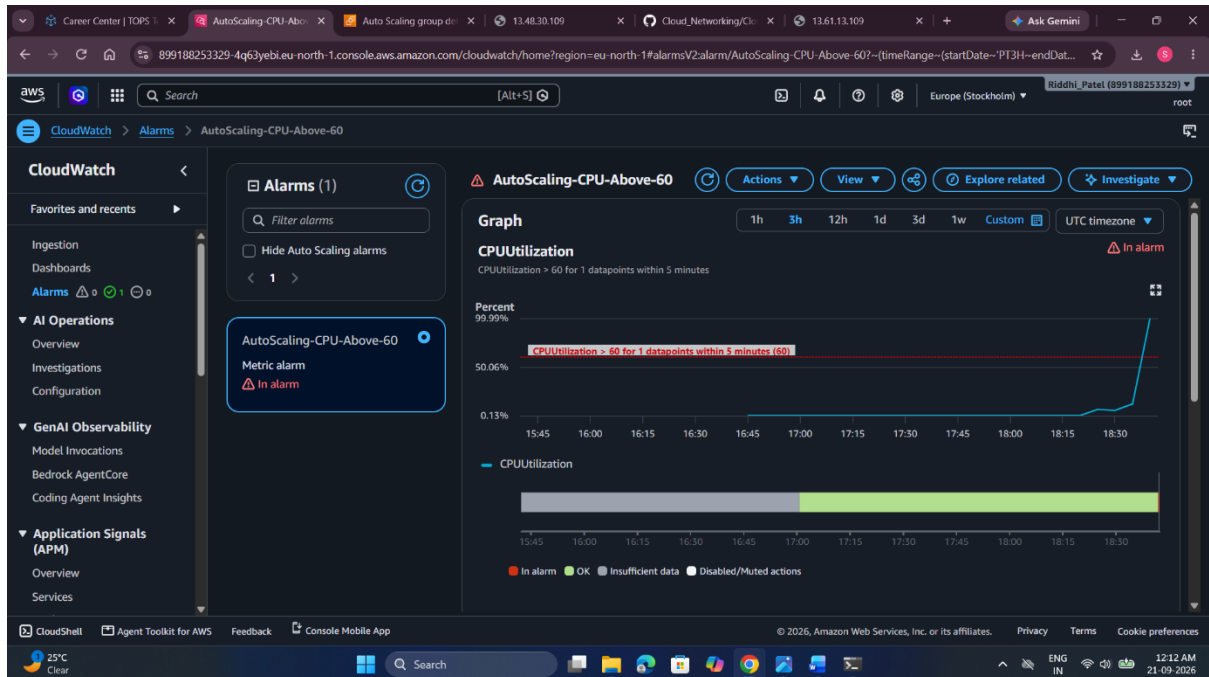
Screenshot 4: CloudWatch CPU Utilization – In Alarm



Step 5:

The CloudWatch alarm AutoScaling-CPU-Above-60 changed from OK to In alarm, which triggered the Auto Scaling policy.

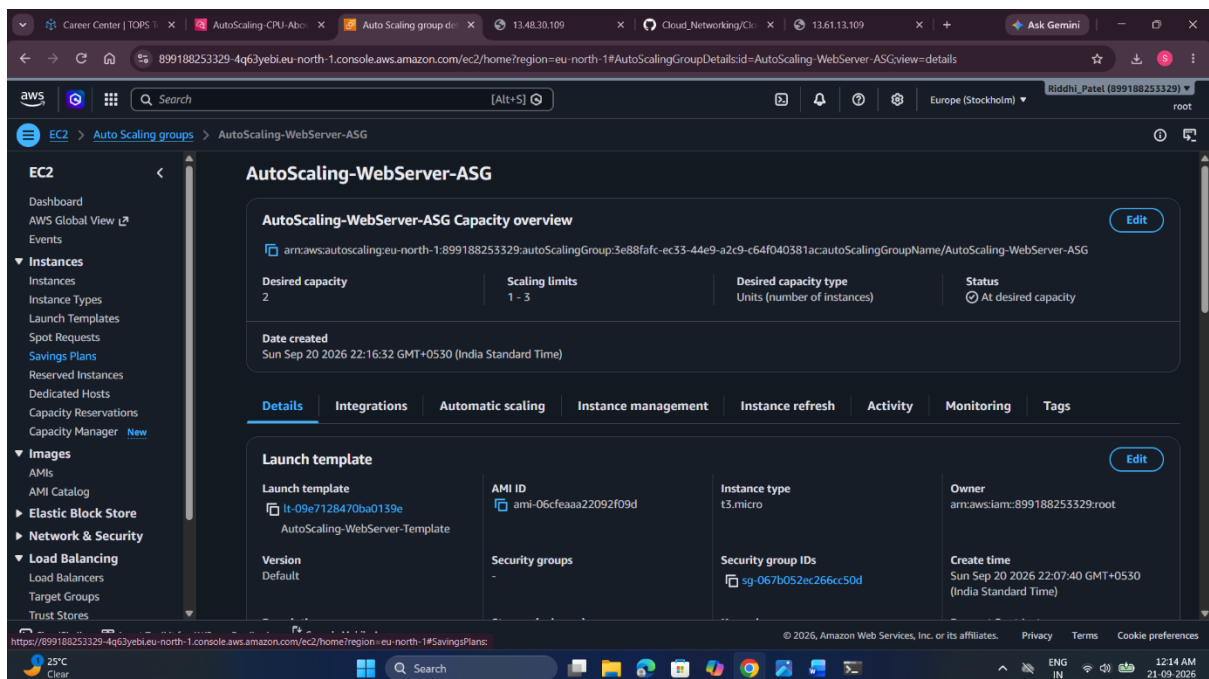
Screenshot 5: CloudWatch Alarm in Alarm State



Step 6:

The Auto Scaling Group automatically increased its desired capacity from **1 to 2** instances.

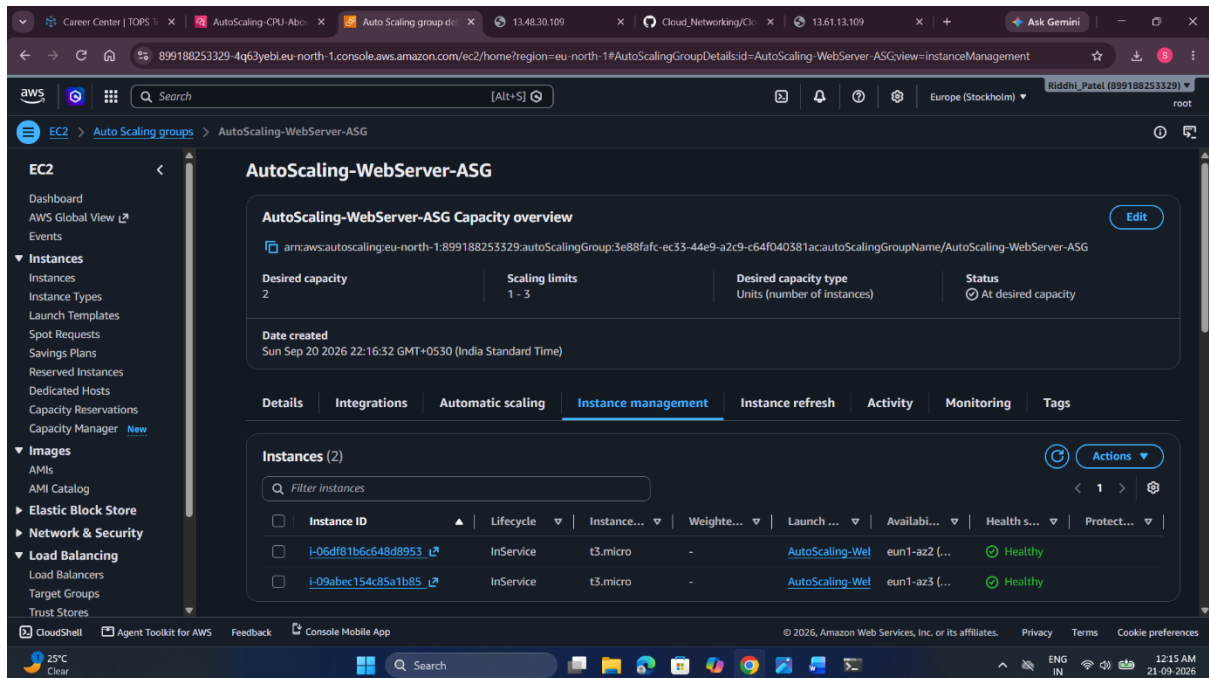
Screenshot 6: Auto Scaling Group Scaled to 2 Instances



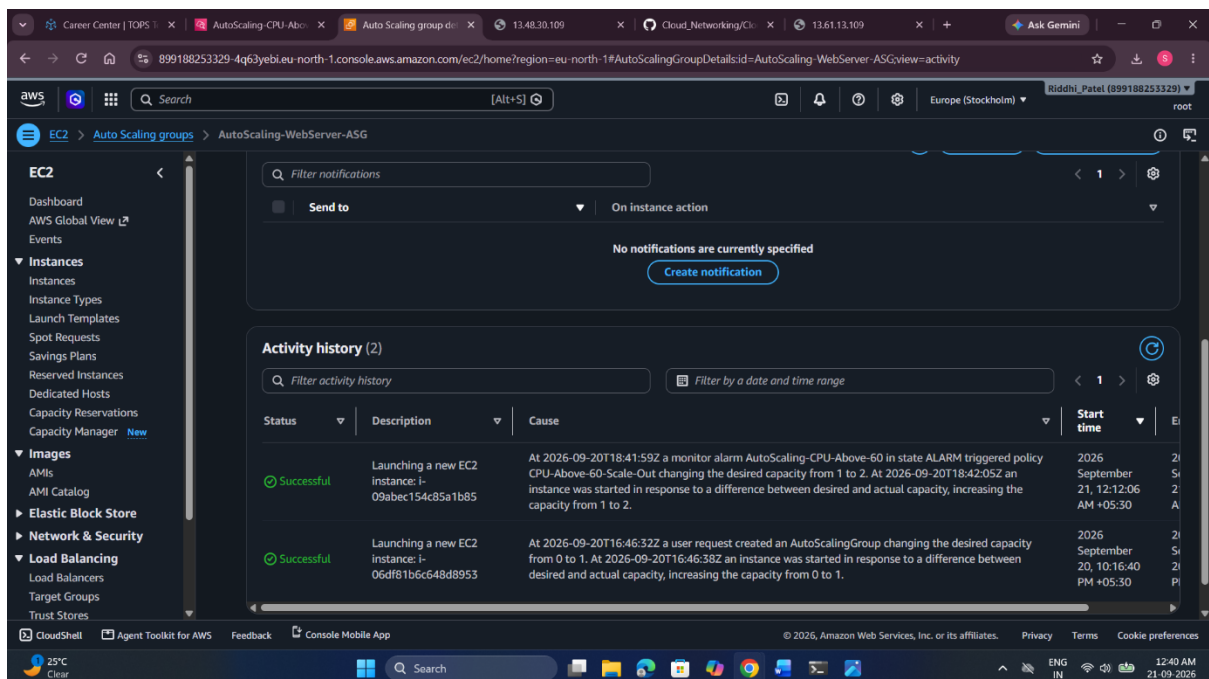
Step 7:

The **Instance Management** section was checked, and both instances were shown as **InService** and **Healthy**.

Screenshot 7: Two Healthy Instances in Auto Scaling Group



Screenshot 7A : Auto Scaling Activity History – New Instance Launched



Result:

The Auto Scaling Group successfully detected the high CPU utilization and automatically launched an additional EC2 instance, increasing the capacity from **1 to 2 instances**.

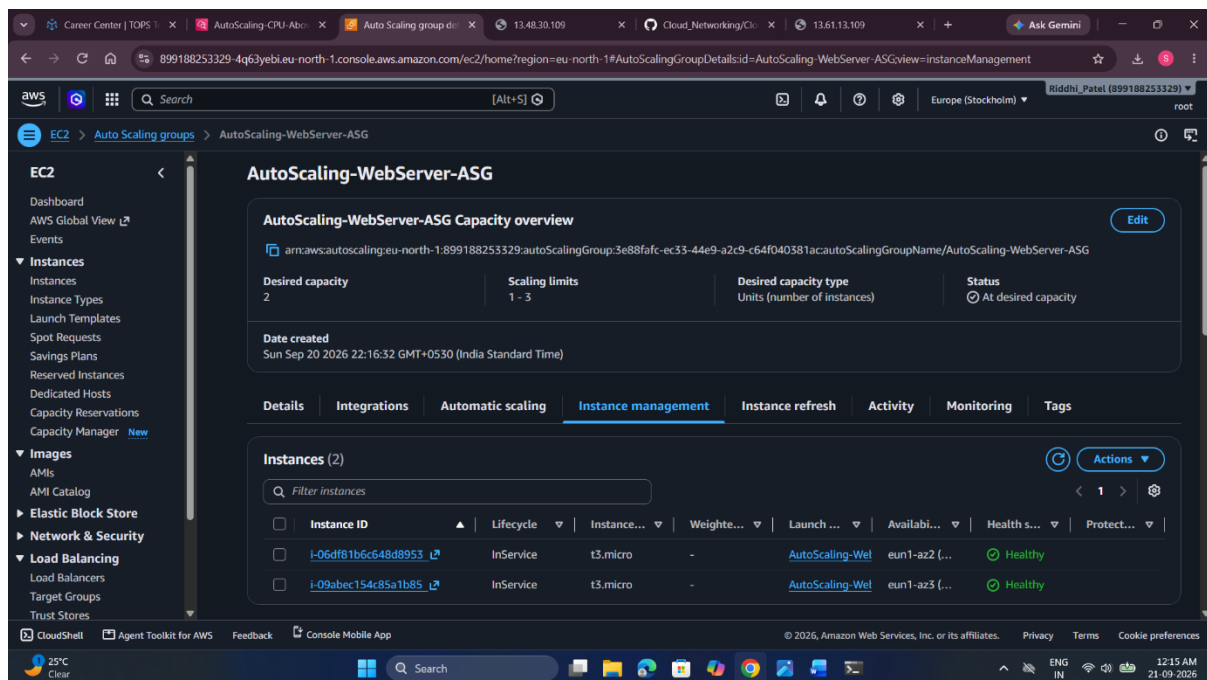
4. After scaling up, reduce the simulated load and verify that the Auto Scaling Group scales the number of instances back down to the minimum. Take screenshots of the scaling activity history as evidence.

Ans: Reduce the Simulated Load and Verify Scale-In

Step 1: Check the Auto Scaling Group after scaling up

After the load test, the Auto Scaling Group had **2 EC2 instances** with the desired capacity set to 2.

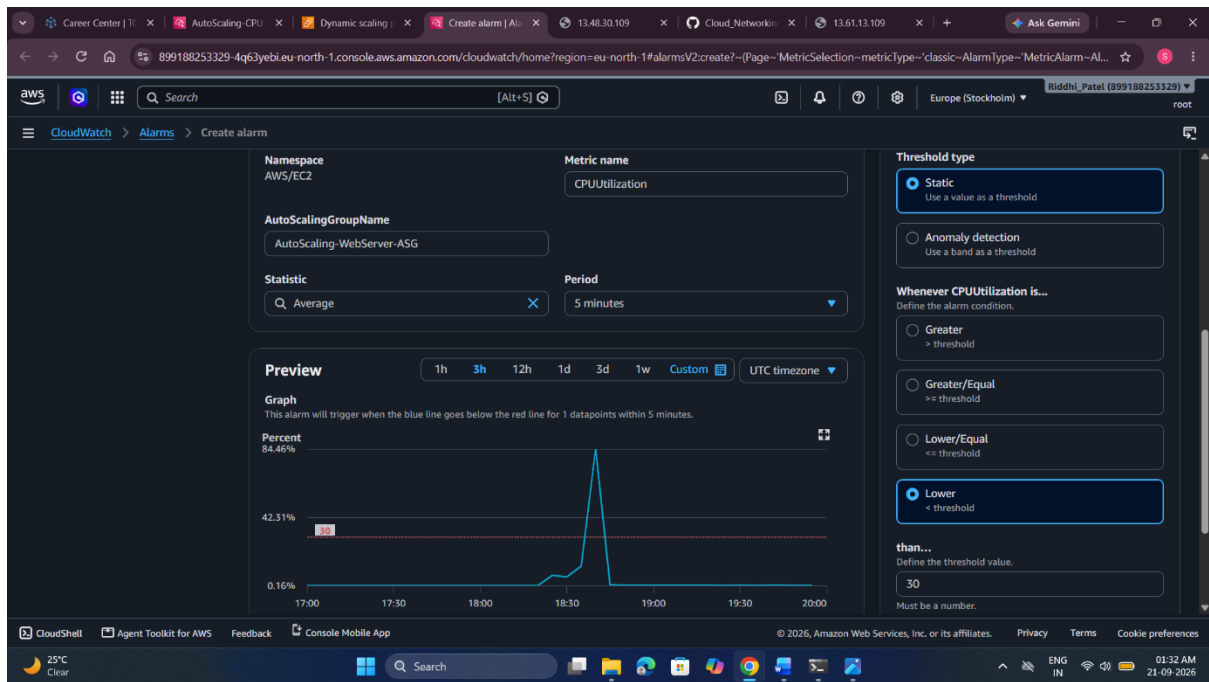
Screenshot 1: Auto Scaling Group with 2 Instances



Step 2: Configure the scale-in alarm

A CloudWatch alarm named **AutoScaling-CPU-Below-30** was configured to trigger when CPU utilization stayed below **30% for 5 minutes**.

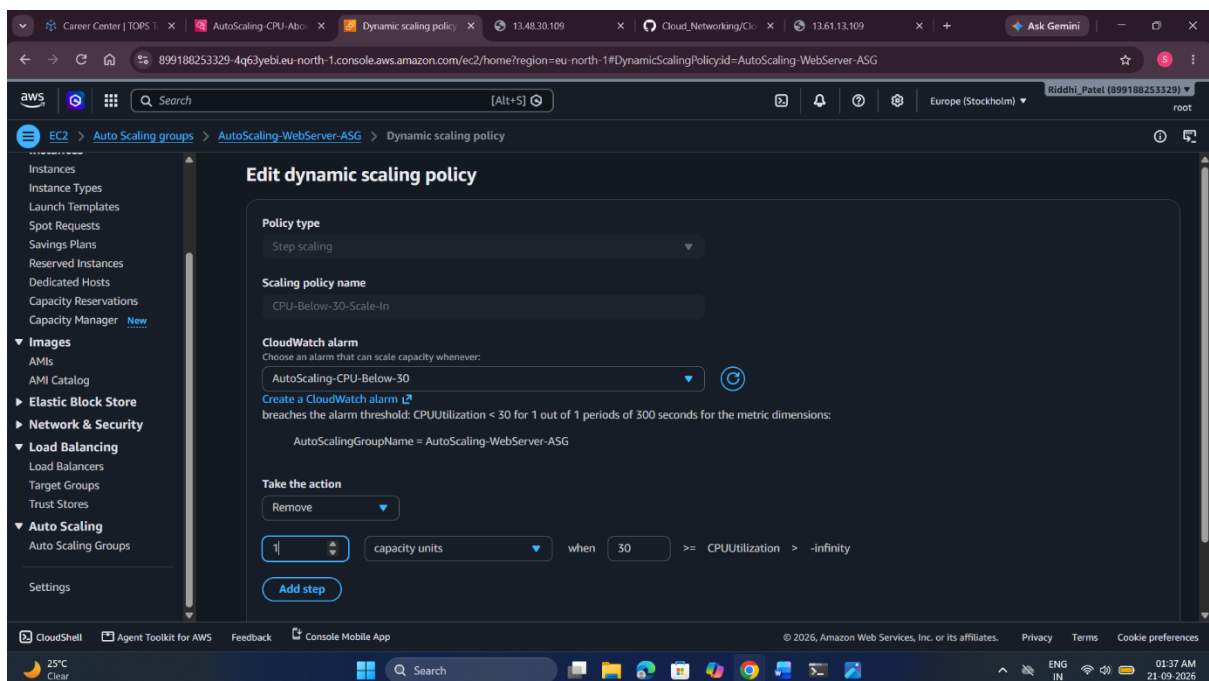
Screenshot 2: CloudWatch Scale-In Alarm



Step 3: Configure the scale-in policy

A step scaling policy named **CPU-Below-30-Scale-In** was configured to **remove 1 instance** when CPU utilization was below 30%.

Screenshot 3: Scale-In Policy Configuration



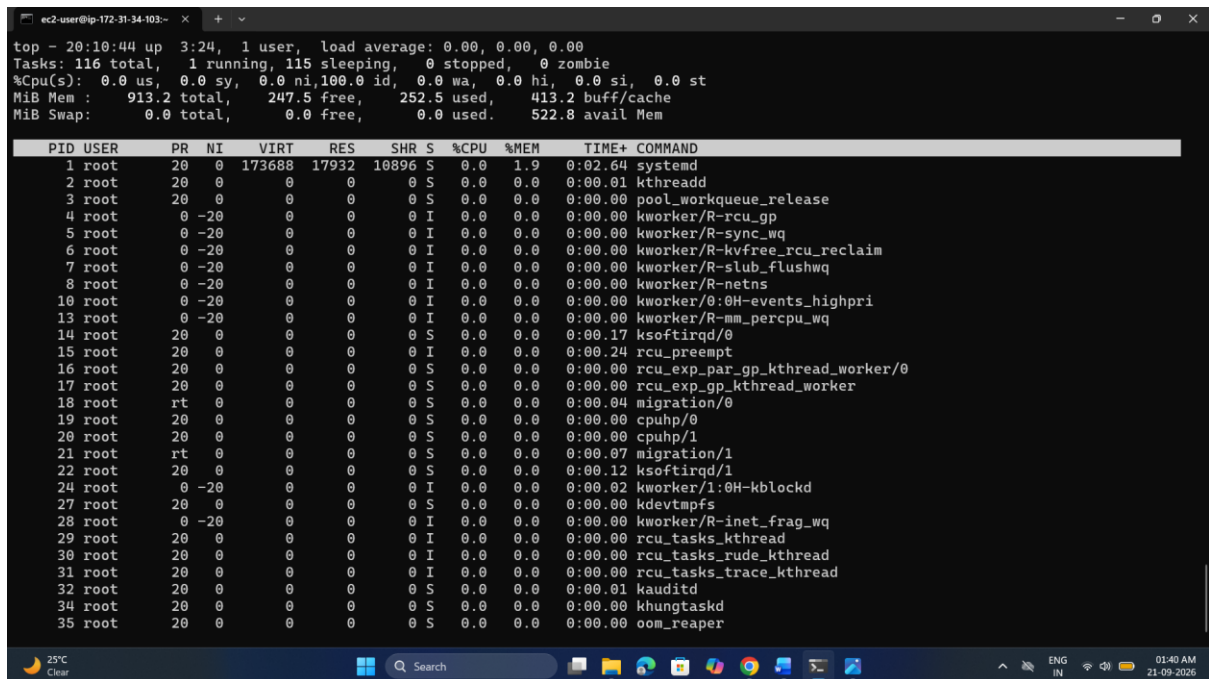
Step 4: Reduce the simulated load

The simulated CPU load was stopped using the command:

pskill yes

The CPU utilization then dropped below the 30% threshold.

Screenshot 4: Reduced CPU Utilization



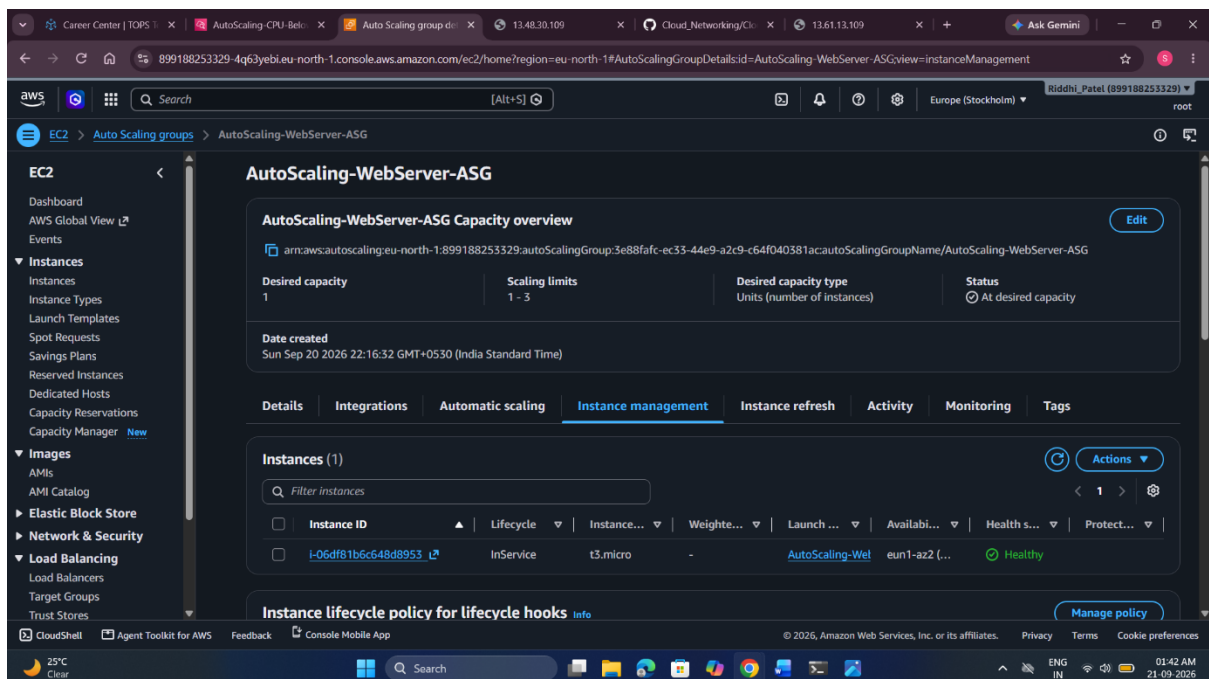
```
ec2-user@ip-172-31-34-103:~$ top
top - 20:10:44 up 3:24, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 116 total, 1 running, 115 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 913.2 total, 247.5 free, 252.5 used, 413.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 522.8 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0 173688 17932 10896 S   0.0   1.9   0:02.64 systemd
    2 root        20   0     0     0     0 S   0.0   0.0   0:00.01 kthreadd
    3 root        20   0     0     0     0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
    5 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
    6 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
    7 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
    8 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-netns
   10 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
   13 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
   14 root        20   0     0     0     0 S   0.0   0.0   0:00.17 ksoftirqd/0
   15 root        20   0     0     0     0 S   0.0   0.0   0:00.24 rcu_preempt
   16 root        20   0     0     0     0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
   17 root        20   0     0     0     0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
   18 root        rt    0     0     0     0 S   0.0   0.0   0:00.04 migration/0
   19 root        20   0     0     0     0 S   0.0   0.0   0:00.00 cpuhp/0
   20 root        20   0     0     0     0 S   0.0   0.0   0:00.00 cpuhp/1
   21 root        rt    0     0     0     0 S   0.0   0.0   0:00.07 migration/1
   22 root        20   0     0     0     0 S   0.0   0.0   0:00.12 ksoftirqd/1
   24 root        0 -20   0     0     0 I   0.0   0.0   0:00.02 kworker/1:0H-kblockd
   27 root        20   0     0     0     0 S   0.0   0.0   0:00.00 kdevtmpfs
   28 root        0 -20   0     0     0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
   29 root        20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
   30 root        20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
   31 root        20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
   32 root        20   0     0     0     0 S   0.0   0.0   0:00.01 kauditd
   34 root        20   0     0     0     0 S   0.0   0.0   0:00.00 khungtaskd
   35 root        20   0     0     0     0 S   0.0   0.0   0:00.00 oom_reaper
```

Step 5: Verify the scale-in

After the CPU remained below the threshold, the Auto Scaling Group automatically reduced the desired capacity from **2 to 1**.

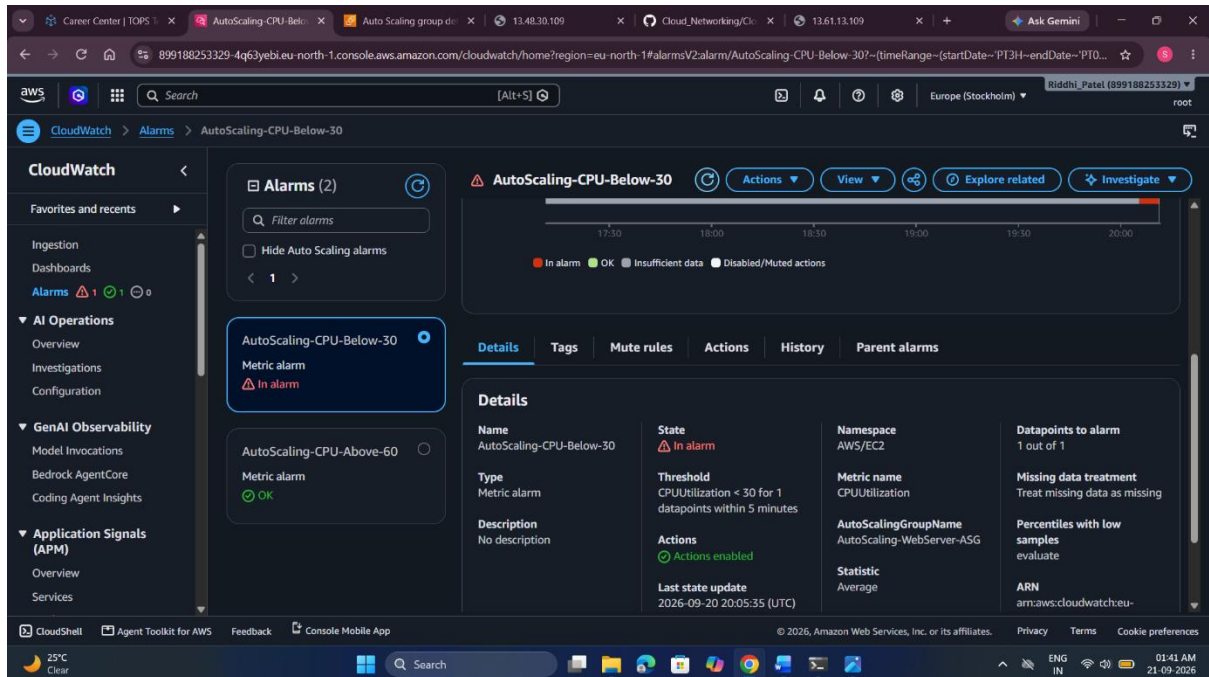
Screenshot 5: Auto Scaling Group with 1 Instance



Step 6:

The CloudWatch alarm AutoScaling-CPU-Below-30-Scale-in changed from OK to In alarm, which triggered the Auto Scaling policy.

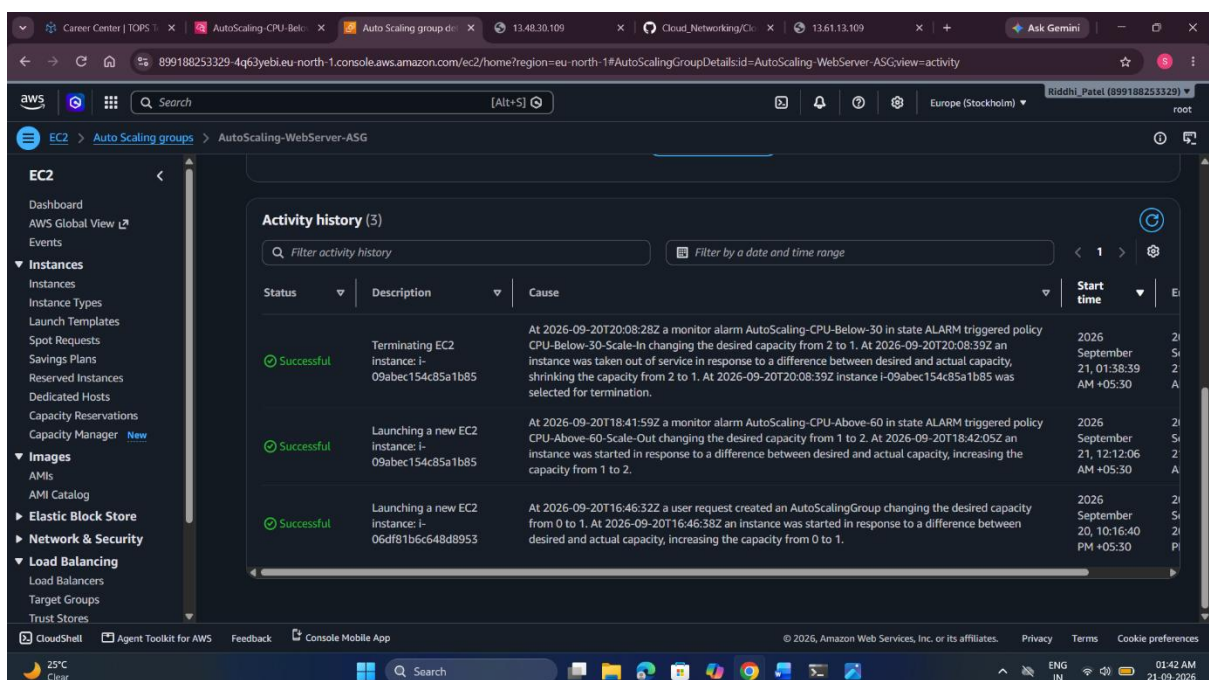
Screenshot 6: CloudWatch Alarm in Alarm State



Step 7: Check the scaling activity history

The Activity History showed a Successful scale-in operation. The AutoScaling-CPU-Below-30 alarm triggered the CPU-Below-30-Scale-In policy and changed the desired capacity from 2 to 1.

Screenshot 7: Auto Scaling Activity History – Successful Scale-In



Result

When the simulated workload was reduced, CPU utilization decreased below 30%. The Auto Scaling Group automatically removed one instance and returned to the minimum capacity of 1 instance.

- 5. Use ChatGPT or another AI tool to generate a list of 3 real-world scenarios (from apps like Zomato, Flipkart, or BookMyShow) where auto-scaling is essential, and explain in your own words why auto-scaling helps in each scenario.**

Ans: Real-World Scenarios Where Auto-Scaling Is Essential

1. Zomato – Peak Lunch and Dinner Hours

During lunch and dinner time, a large number of users order food at the same time.

How Auto-Scaling Helps:

Auto-scaling automatically adds more servers when the number of users and requests increases. When the traffic decreases, it removes the extra servers. This helps Zomato handle orders smoothly and maintain good performance.

2. Flipkart – Big Sale Events

During major sale events, millions of users may visit Flipkart and place orders at the same time.

How Auto-Scaling Helps:

Auto-scaling launches additional servers to handle the sudden increase in traffic. This prevents the website from becoming slow or unavailable. After the sale, extra servers can be removed when traffic decreases.

3. BookMyShow – Popular Movie Ticket Booking

When tickets for a popular movie are released, many users may try to book tickets simultaneously.

How Auto-Scaling Helps:

Auto-scaling increases the number of servers when booking requests suddenly increase. This helps the application handle more users and reduces slowdowns or failures during peak booking time.